https://hjfy.top/arxiv/2610.12424

NOEMATRIX

RoboRSI：在复杂真实环境中的稳定、高效且可重用的机器人自我演化

Zimo Wen*, Yijin Chen*, Yuxuan Cao*, Wendi Chen, Yanwen Zou, Wenye Yu, Fuhang Kuang, Han Xue, Jun Lv, Chuan Wen†, Cewu Lu†

Abstract

在复杂真实环境中的通用机器人不仅应能执行多样化任务，还应通过经验不断改进，将执行过程中学到的知识转化为后续任务可复用的能力。通过代码行动的机器人智能体已具备从执行反馈中修复程序的能力，但如何围绕赋予其意义的任务结构来组织这些经验，仍是一个核心挑战：确保每次修复都能归因于相应的功能能力，由执行证据支持，并在复用前经过验证。我们提出了RoboRSI，一种基于自顶向下技能精炼（TSR）的机器人自我改进系统。TSR 将任务分解为具有明确职责范围的复合、原子和基础技能，并定义清晰的输入-输出契约；将每次执行结果归因于相应的分支，并将修改限制在该分支内。在此结构基础上，经理、规划器、工程师和评审员协同完成规划、执行、诊断及新技能的验证发布，而人类则通过目标设定和修正引导整个过程；稳定的技能序列进一步整合为可复用的复合技能。在移动操作臂上,RoboRSI 历经 104 轮迭代，实现了多目标家庭清洁任务。在仿真环境中，其在 LIBERO、LIBERO-PRO、LIBERO-Plus 和 RoboTwin 上的成功率最高，优于最强基准 2.7 至 11.0 个百分点。

项目页面： https://lab.noematrix.ai/blog/2-roborsi-research-preview/ **代码：** https://github.com/nssmd/RoboRSI

Keywords: Robot self-improvement, code as policy, multi-agent systems, skill learning, mobile manipulation

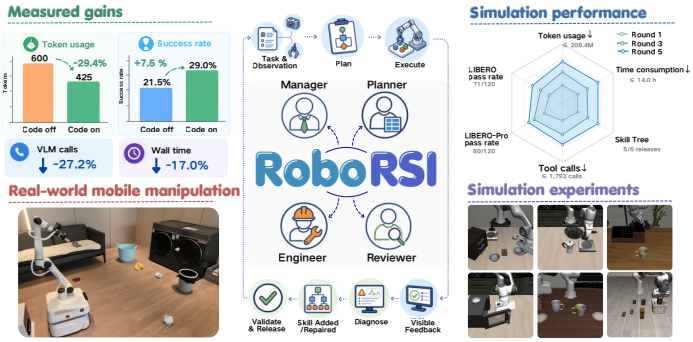


Figure 1 | **RoboRSI 一瞥。**多智能体自提升环（中心），真实世界中的移动操作与复合技能提升（左侧），以及仿真结果（右侧）。

*Equal contribution. †Corresponding authors.

Contents

1 引言 **4**

2 相关工作 **5**

3 方法 **6**

3.1 总体框架 6

3.2 自顶向下技能优化 7

3.3 多智能体系统 9

4 实验 **9**

4.1 实体机器人案例研究 9

4.2 评估设计 10

4.3 主要仿真结果 11

4.3.1 LIBERO 套件结果 13

4.3.2 一日自我迭代 15

4.4 案例研究 15

4.5 消融研究 16

4.5.1 角色分离 17

4.5.2 自顶向下技能优化 17

4.5.3 代码合并 18

4.5.4 骨干模型 18

4.5.5 从执行数据中学习策略 19

5 结论与未来工作 **20**

A TSR 配方与分析 **26**

B 物理评估情景 & 评估流程 **27**

C 补充实验 **28**

C.1 故障分类 28
C.2 探索性开发 29
C.3 从执行数据中学成策略 29

D 案例研究：技能代码与故障驱动的修订 29

D.1 执行修复：消除过早的目标排斥 29
D.2 生成复合技能：验证、发表和调用 31
D.3 代码前后对比：由评审驱动的终端检查 31
D.4 第 25 轮和第 33 轮的物理技能修订 33
D.5 需要人类知识的更正 34

1. 引言

通用机器人技术的愿景是开发出能够在复杂多变的真实环境中可靠执行多种任务的机器人, 这些环境中的物体、布局和指令不断变化, 同时能够通过经验不断改进。这使得机器人智能体的目标从回合级别的任务完成, 转向持续的能力积累：失败的抓取、过时的物体位置以及恢复情境只有在机器人能够识别所涉及的能力, 利用执行证据进行优化, 并将改进后的行为应用于后续任务时才具有价值。

一种有前景的路径使得基础模型能够通过代码进行操作。代码即策略 (Liang et al., 2023) 将感知和控制原语组合成可执行的机器人程序, 而 CaP-X (Fu et al., 2026) 研究抽象、具身化与交互如何塑造此类程序。后续工作通过执行反馈实现闭环：ETA/OpenETA (Chen et al., 2026b) 在规划与工具执行之间交替进行, 并结合新的观测结果,ASPIRE (Lu et al., 2026) 将修复后的程序加入不断增长的技能库中, 而 ENPIRE (Xiao et al., 2026) 则将这一环路延伸至物理实验。因此, 执行反馈的作用从修正单次尝试的信号, 转变为构建跨任务持续存在的能力资源。

然而, 只有当执行经验围绕赋予其意义的任务结构进行组织时, 才能成为可复用的能力。修复必须始终与推动它的目标保持关联, 属于负责观察到的行为的能力范畴, 并在重用前经过验证。若缺乏这种结构, 重复尝试会在上下文中填满完整的轨迹, 局部修复会偏离整体目标, 人们也会被反复拉回重新设置环境、查阅日志以及判断每个修复应归属何处的循环中。

我们的观察是, 这种结构必须同时服务于指挥机器人的人员以及实现其行为的智能体, 这需要两种互补的抽象。*面向人类的引导*使人们能够在所需服务的层面表达目标、领域知识、约束和安全判断。*面向智能体的结构*为智能体提供了任务相关的上下文、明确的接口以及执行与修订的有限责任。

基于这一观察, 我们引入了 **RoboRSI**, 一个由 **自顶向下技能精炼 (TSR)** 组织的 **多智能体系统**, 用于机器人自我改进。TSR 将每个任务分解为具有范围职责和明确输入–输出契约的复合技能、原子技能和基础技能, 将每次执行结果归因于相应的责任分支, 并将修改限制在该分支内：分解不当则修正任务结构, 组合不可靠则修正相应复合技能, 操作失败则修正负责的原子或基础技能。管理者、规划者、工程师和评审员分别负责规划、执行、诊断和有效发布, 稳定的技能序列被整合为复合技能, 后续执行将选择并监控这些复合技能, 而非每次都重新构建每个工具决策。

我们在移动操作机器人上评估了 RoboRSI, 该机器人在场景变化下执行多物体家庭清洁任务, 其技能在 104 轮训练中逐步发展, 并在仿真环境中与机器人智能体基准方法在 LIBERO、LIBERO-PRO 和 RoboTwin 上的表现进行对比, 同时将冻结能力迁移至 LIBERO-Plus。RoboRSI 在所有四个基准测试中均取得了最高的成功率, 在 LIBERO 上超越最强基准 5.3 个百分点, 在 LIBERO-PRO 上超出 11.0 个百分点, 在 LIBERO-Plus 上超出 5.7 个百分点, 在 RoboTwin 上超出 2.7 个百分点。

我们的贡献体现在三个方面：

- **机器人自我改进的互补抽象：**我们发现, 只有当执行经验与赋予其意义的任务结构相联系

时, 这些经验才能被重复利用, 且该结构必须同时服务于人类友好的引导和智能体友好的结构。

- **RoboRSI 框架：**一种通过自顶向下技能精炼实现这些抽象的系统, 该系统在每个原子任务完成前验证其后条件, 并在最早出现失败的结点处修正每次失败；以及一个跨任务验证和发布技能的多智能体执行-精炼环。
- **真实世界与仿真评估：**在 104 轮实验中, 移动操作臂的技能发展情况, 以及在四个仿真基准上的最高成功率, 还有角色分离、技能精炼和复合技能的消融实验。

2. 相关工作

代码即策略。 代码即策略 (Liang et al., 2023) 表明, 语言模型可以编写组合感知与控制 API 的机器人程序, 将高层指令转化为可执行行为。后续工作探讨了这些程序应如何构建：CaP-X (Fu et al., 2026) 研究了基本操作的抽象层次、感知基础以及交互方式,GRACE (Sun et al., 2025) 将可用性与几何信息编码为可执行概念。ASPIRE (Lu et al., 2026) 更进一步, 能够修复失败的程序, 并将修复后的技能存入不断增长的技能库中。一旦程序被保存并重复使用, 对某一技能的修复可能影响所有调用该技能的任务, 因此决定修复应位于何处, 与编写修复本身同样重要。RoboRSI 通过 TSR 解决这一问题, 为每个能力提供接口和明确的责任声明, 使得修复在负责的分支上进行, 并能与周围的技能良好组合。

递归自我改进（RSI）。 RSI 研究智能体的经验如何提升其后续行为。语言智能体最初通过言语反馈实现这一目标：Reflexion 会存储对失败尝试的反思,Self-Refine 则迭代地批判并修正自身的输出 (Madaan et al., 2023; Shinn et al., 2023)。Voyager 从文本转向可执行技能, 通过开放式交互积累代码库 (Wang et al., 2024)。多模态智能体则进一步拓展这一理念, 通过外部资源获取的技能、执行反馈以及可复用的流程, 如 RESOURCE2SKILL、EvoSkill-GUI 和 OmniHarness (Chen et al., 2026a; Fan et al., 2026; Xu et al., 2026) 所示。

针对机器人特定的研究在执行过程中检测并解释失败：ProgPrompt 在生成的程序中加入断言 (Singh et al., 2023),REFLECT 将机器人经验进行总结以解释并纠正失败 (Liu et al., 2023b),DoReMi 与 Code-as-Monitor 监控由语言或代码指定的约束条件, 以触发恢复机制 (Guo et al., 2024; Zhou et al., 2025), 而 AHA 训练一个视觉-语言模型来检测并推理操作失败 (Duan et al., 2025)。这些方法在回合内检测失败；而 RoboRSI 进一步决定应修改哪项技能, 并在后续任务重用前验证该修改的有效性。

在具身情景中,EmbodiSkill 在单个技能层面诊断错误 (Ju et al., 2026),ENPIRE 将物理评估与编码智能体对策略的优化相结合 (Xiao et al., 2026)。物理执行带来了软件环境通常避免的挑战：一次失败可能源于数步之前, 且对共享技能的修改会影响未参与失败运行的任务。因此,RoboRSI 将每个失败归因于其最早的失效结点, 基于受影响任务的近期历史验证修改, 并通过显式的技能接口重用能力。

用于机器人的大型语言模型 大型语言模型最初作为规划器进入机器人领域。早期工作将生成的计划建立在可接受的动作之上 (Huang et al., 2022a), 而内部独白（Inner Monologue）则将环境反馈重新引入规划过程 (Huang et al., 2022b)。随着多模态理解 (Alayrac et al., 2022; Gemini Team et al., 2024; Liu et al., 2023a)、具身定位 (Driess et al., 2023) 以及推理与动作交错进行 (Yao et al., 2023) 的进展, 模型得以直接基于观测进行推理,VoxPoser (Huang et al., 2023) 和 ReKep (Huang et al., 2024) 利用生成的代码组合出三维价值图和关系关键点约束以用于运动规划。另一条研究路径则采用端到端方式训练机器人基础模型。视觉-语言-动作模型如 RT-2、OpenVLA、π_0 以及 $\pi_{0.5}$ 从大规模机器人数据集中学习视觉运动控制 (Black et al., 2025; Brohan et al., 2023; Kim et al., 2024; Physical Intelligence et al., 2025)；世界-动作模型如 Zero-WAM 增加了预测性视觉动态 (Zhou et al., 2026)；Gemini Robotics 与 Gemini Robotics-ER 1.5 将控制与具身推理相结合 (Gemini Robotics Team et al., 2025)。随着通用模型能力的不断提升, 近期系统开始直接使用它们做出机器人决策与动作。Maestro 协调感知与控制模块 (Shi et al., 2025),ETA/OpenETA 实现了规划、执行与反馈之间的闭环 (Chen et al., 2026b),GPT-Policy 实现上下文内控制 (Cheng et al., 2026), 两个 RoboHarness 系统通过记忆实现自适应 (Huang et al., 2026; Li et al., 2026),UniDomain 与 UniPlan 将符号规划与视觉定位相连接 (Ye et al., 2025, 2026)。RoboRSI 在系统层面补充了这些方向：TSR 与多智能体执行维持可重用技能, 执行过程中收集的轨迹可用于训练返回作为同一层级工具的策略。

3. 方法

考虑一个直接连接到机器人工具的编码智能体, 在多次运行中被要求改进复杂现实家庭环境中的家务清理任务。上下文充满了之前运行的记录；例如, 序列后期观察到的失败会在其出现位置进行修复, 如在释放步骤中增加重试机制, 尽管问题的根源在于感知或导航；并且由于目标隐含在最新的提示中, 后续的修改会优化最新场景。人们因此不得不反复查阅日志并判断每个修复应放在何处。RoboRSI 通过使人类意图和智能体工作都变得明确, 消除了这一负担。

3.1. 总体框架

RoboRSI 将重复的机器人执行转化为一个围绕第 1 节中两个抽象组织的可重用技能库。**人类友好的引导**是人们以任务层面输入目标、约束、领域知识和修正信息的通道；每次修正都会作为对相关技能的持续指导予以保留。**智能体友好的结构**是智能体工作的任务-技能层次结构：每个任务和技能都具有接口、声明的责任以及最近执行记录, 因此每个智能体仅接收当前步骤所涉及的层次结构部分。

技能等级体系。 该层级包含四种结点（图 2 和 3）。一个 *任务族*, 例如家庭清洁, 将具有共同目标和约束的任务归为一类。每个任务被分解为 *原子任务*, 即具有明确作用范围的目标, 且可观察到结果, 如将物体放入容器中, 每个原子任务由一个 *原子技能*实现, 该技能具有后置条件。原子技能调用 *基础技能*, 即作为工具暴露的感知与控制原语。一组稳定的技能序列被整合为 *复合技能*, 这是一种参数化的代码策略, 以单一单元被调用。每个技能声明其输入、输出及职责, 且一个

技能可被多个分支共享。

图 2 展示了一个发展回合：智能体对既定目标进行分解，并规划已释放的技能，将其执行至基础技能，从可见证据和工具迹中定位最早失败的结点，验证并发布后续更新以进入下一回合。人们在第一阶段以及每次需要修正时进行引导。

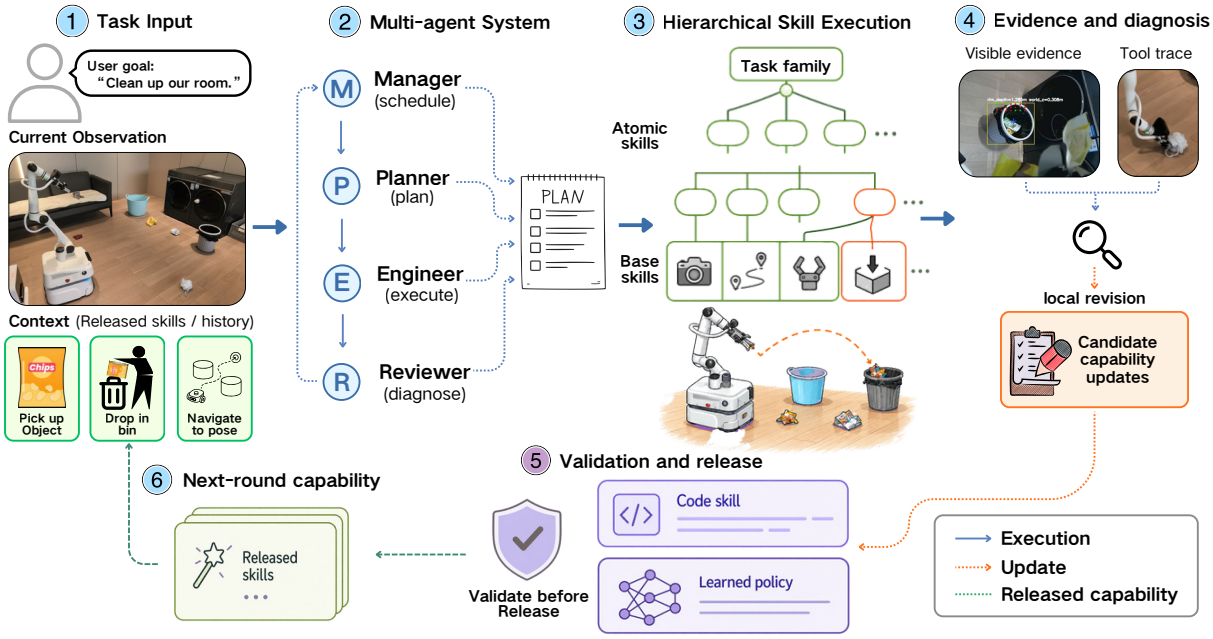
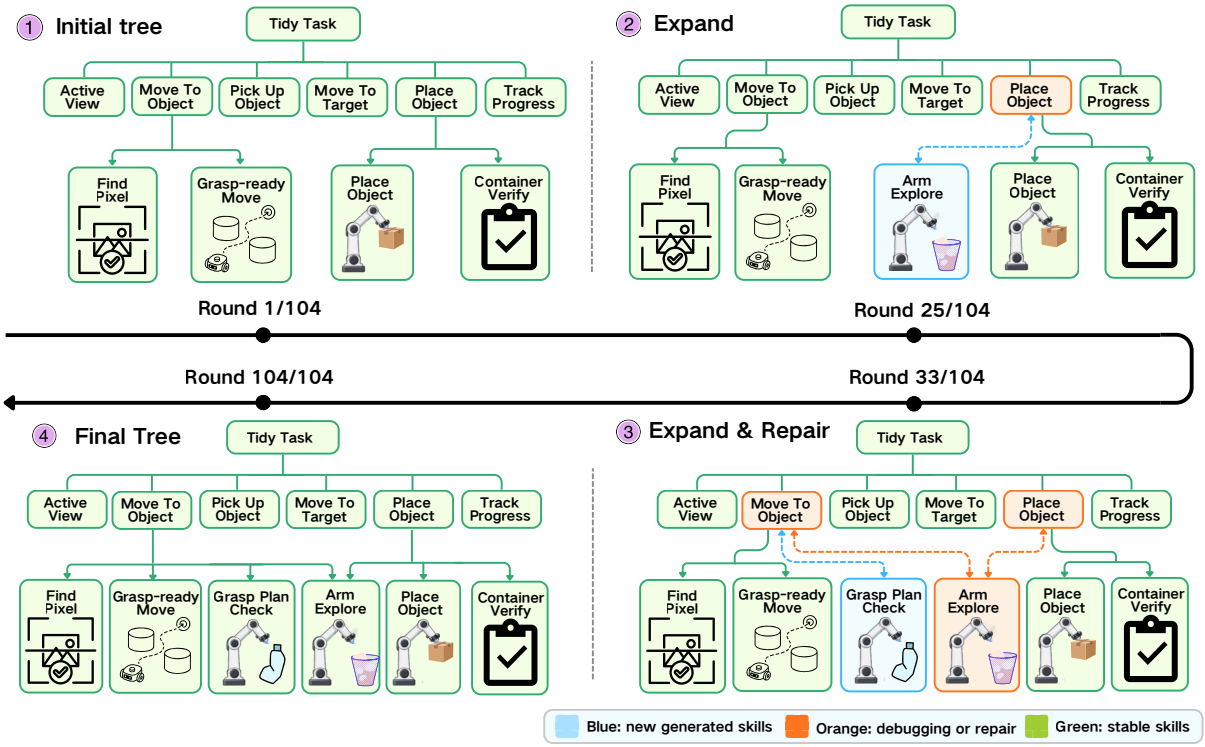


Figure 2 | **RoboRSI 框架**。给定一个任务，管理者组织其技能层级，规划者生成执行计划，工程师运行所需技能。评审者从观测结果和工具迹中诊断失败情况，并将其返回以进行基于 TSR 的修正。经过测试的更新进入下一次迭代，而稳定的分支则成为后续任务的参数化技能。

3.2. 自顶向下技能优化

编码智能体可以解决局部编程问题，但在反复修改过程中会丢失任务目标，误解物理条件，或过于针对某一失败场景进行修复。因此,TSR 围绕第 3.1 节中的技能层级组织实现与修订：任务族及其原子任务保持全局目标不变，而原子、复合和基础技能则为实现、经验积累和修订提供了有界的位置（图 3a）；图 3b 展示了对物理机器人进行两次修订的过程。

设计原则。 三个原则将局部修订与可靠执行和复用联系起来。*责任与接口信任*：每个技能通过显式接口报告其结果，因此调用者依赖其后条件，而非重复检查其正确性，诊断也沿着调用路径定位到负责的组件，而非在调用者之间分散修复。*参数化与历史引导的修订*：对象类、条件、目标关系和执行顺序成为参数，场景几何来自当前观测，每次变更都会与该技能最近的成功使用进行比对验证。*持久的人类知识*：人类修正变为运行时检查、测试或对负责技能的持续指导，使人们能够专注于目标和探索方向。



(a) Task–skill structures during physical task development



(i) Item held at the bin (ii) On-board view of the bin opening (iii) After revision: before / after release

(iv) Chassis view: bucket at image edge (v) In-hand view: bucket fully visible (vi) Run ends with the item held

(b) Two TSR revisions on the physical robot

Figure 3 | **物理清理过程中的 TSR**。(a) 四个代表性任务-技能结构展示了从初始任务提议到技能库逐步发展的过程；彩色块标记了每个快照中的技能操作。(b) 第 25 轮（上方）：机器人将包装材料持在正确料箱位置但未释放（i），尽管机械臂已到达观测姿态（ii）；运动调用报告了容差失败，且回退机制耗尽其预算。TSR 将失败归因于该观测动作，并仅修正“放置物体”分支，重新检查到达姿态并执行完整的代码生成释放序列；下一次放置成功（iii）。第 33 轮（下方）：目标料箱在底盘视角中仅出现在图像边缘（iv），但在同一扫描的手中视角中完全可见（v）；消费者拒绝了生产者的终点位置，运行以第二个物品仍被持有而结束（vi）。修正策略保留了生产者身份，并优先选择完整视图。两次修正均通过了最近四次运行的离线重播及现有回归测试（附录 D.4）。

精炼步骤。 我们将第 t 轮释放的层次结构表示为一个图 $G_t = (V_t, E_t)$, 其中结点为任务和技能, 边表示可能的调用; 每个结点 v 携带一个实现或指导 c_v 以及一个声明的后置条件 ϕ_v。执行一个任务会产生一个记录 τ_t, 包含观测结果和工具调用。我们将任务分支上执行过的结点按其返回顺序列出为 $\pi_t = (v_1, \ldots, v_m)$, 使得每个被调用者都位于其调用者之前; 若某结点被重试, 则每次尝试都单独列出。评审者沿该路径评估后置条件, 并返回第一个失败的结点。

$$v_t^\star = v_{k^\star}, \qquad k^\star = \min\{k : \phi_{v_k}(\tau_t) = 0\}. \tag{1}$$

修订范围 $S_t \subseteq V_t$ 是由 $v_t^\star$ 拥有的子层次结构,∂S_t 表示其与 G_t 其余部分的接口。随后进行一次修订回合

$$\Delta_t = \mathcal{R}\big(G_t|_{S_t}, \partial S_t, \tau_t\big), \qquad G_{t+1} = G_t \oplus \mathcal{V}(\Delta_t; \mathcal{H}_t). \tag{2}$$

此处 $\mathcal{R}$ 为审稿人的补丁提议, 仅可修改 S_t 中的结点; 若接口本身必须更改, 则首先将作用范围扩展至其调用者。$\mathcal{H}_t$ 收集所有调用图可达 S_t 的任务的最近失败与成功执行记录。验证算子 $\mathcal{V}$ 在通过审查、功能测试以及在 $\mathcal{H}_t$ 上重放且无回归的情况下返回 Δ_t, 否则返回空补丁; $\oplus$ 应用已接受的补丁。公式 (1) 将每个修复置于故障的原点而非观测位置, 对 S_t 的限制将影响范围限定在可达 S_t 的任务, 而对 $\mathcal{H}_t$ 的验证则保留已有正常行为（附录 A）。

从稳定分支到可复用的能力。 稳定分支演变为复合技能, 使在线推理从单一工具选择转变为技能的选择与监控; 执行轨迹也可用于训练处于同一层级中的作为技能暴露的神经策略（第 4.5.5 节）。

3.3. 多智能体系统

四个智能体在此结构中执行每一轮。**管理器**将目标分解为原子任务, 识别每个任务所需的基础技能, 维护技能版本, 审查每个提出的修改建议, 可请求或撰写修改, 并决定是否接纳该修改、其可更改的分支以及是否发布。**规划者**接收一个原子任务、当前观测结果以及其下层技能的接口, 返回一个可执行的计划。**工程师**通过工具接口执行计划, 并实现缺失的技能。**评审者**根据观测结果和工具迹判断原子任务是否达到预期结果; 若失败, 则识别执行首次发散的位置, 并为负责的分支编写修复补丁提交给管理器。每个智能体仅处理其步骤所涉及的层次部分, 因此其上下文保持简短, 且每次变更均可追溯至诊断结果。

4. 实验

我们报告了物理部署和仿真结果, 随后对比了各个回合并总结了消融实验。

4.1. 实体机器人案例研究

情景 我们使用 WheelSingleArm M1 移动机械臂研究家庭地板清理任务。该机器人会搜索物体, 靠近并抓取它们, 然后将其运送到指定的容器中。案例研究包含 104 次物理开发运行, 累计运行

时长为 24 小时。每次运行前, 场景都会恢复到一个通用的基础布局, 而物体和容器的位置则部分随机化；导航地图保持不变。平台细节和重置流程见附录 B。

迭代清理。 图 4 展示了 104 次运行开发周期中选取的 10 次运行。在最早选取的运行中, 机器人未放置任何物体。后续的运行逐渐发展为放置多个物体, 部分运行完成了指定的清理任务并返回停靠站。这一变化并非持续稳定：在较晚的某次运行中, 尽管此前已出现完整的运行记录, 却仅记录到一次放置。我们仅在现场人员观察到物体进入指定容器时才将其计入。该图每轮最多显示四次放置, 尽管任务的实际大小存在差异。

图 3a 中的技能树快照为这一发展过程提供了补充视角。层次结构在 *放置物体* 下引入了 *机械臂探索*, 以获得目标容器的视图。后续的修订在 *移动到物体* 下增加了 *抓取规划检查*, 并使 *机械臂探索* 可用于接近和放置两个分支。这些变化针对清理流程中反复出现的两个需求：在运动前检查候选抓取方案, 以及基于对目的地的观测来确定放置位置。这些快照记录了相应能力的组织与复用方式；所选的执行实例并未隔离单个修订的影响。

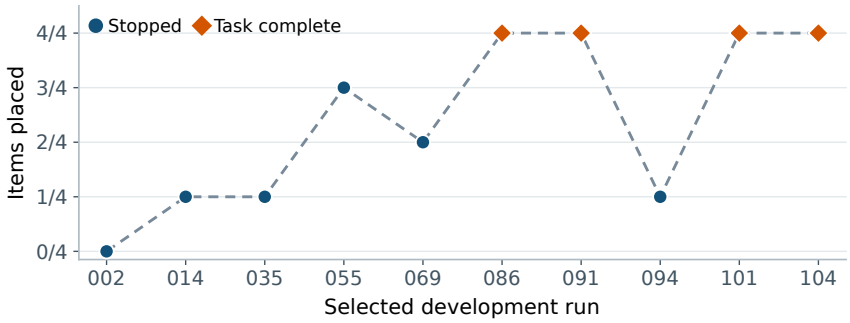


Figure 4 | **选定物理清理运行中的放置进度**。每个点表示在该次运行中前四项考虑的物品中已放置的物品数量；放置已现场确认。标记颜色表示审核后的任务结果。点在选定运行顺序中等距分布, 而刻度标签则给出开发运行编号。虚线段仅连接选定的观测值。四项的分母是一个常见的显示参考：任务清单和物体配置在各次运行中有所不同。

物理执行。 两场景演示提供了一个扩展的执行示例。机器人在第一场景中放置了所有五个指定物体, 在第二场景中放置了全部四个物体, 每次放置均在现场得到确认。场景之间, 操作员重新整理了物体和容器；技能代码、提示信息和策略配置均未修改。清理执行过程无需人工修正或重启。场景切换后, 机器人重复了搜索、靠近、抓取、运输和放置的流程。图 5 展示了此次记录执行的各个阶段。

4.2. 评估设计

基准选择。 RoboRSI 以串行方式累积技能：每个回合均使用当前的技能库运行, 通过验证的修订将被发布给后续的回合。据我们所知, 目前尚无其他机器人智能体系统以这种串行形式累积技能。ASPIRE (Lu et al., 2026) 是最接近的自我改进代码智能体, 它并行学习每一项任务, 由一个专用智能体在候选程序集合上运行基于种群的搜索, 接受搜索结果中的技能, 并为每项任务评估

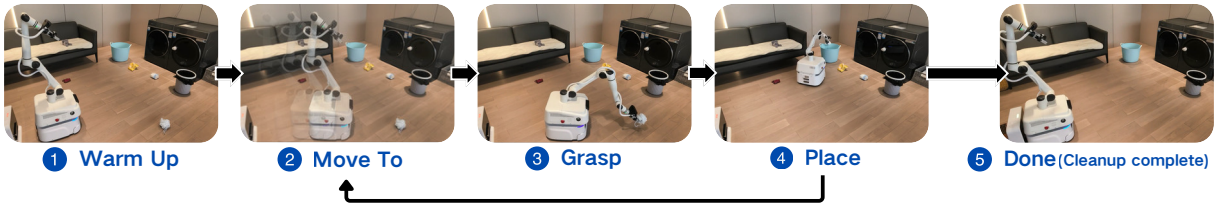


Figure 5 | **两场景物理移动清理**。选定的帧展示了准备、接近、抓取、放置以及记录的最终场景。返回箭头表示重复的物体操作。物体和容器在场景间被重新排列，而智能体配置保持不变。

一个生成的程序；ENPIRE (Xiao et al., 2026) 则通过物理实验来优化学成的策略及其训练代码。两者都依赖于与单一串行流不同的开发环，因此无法在相同协议下运行。因此，我们与通过代码或工具行动的最强智能体 CaP-X、Maestro 和 OpenETA 进行比较，它们采用相同的骨干架构。

自我提升作为评估的能力。 所有方法均在相同条件下进行评估：使用相同的主干网络、工具接口、每回合交互预算、任务面板和模拟器判定标准。基准方法使用一组固定的工具和技能，并且在各回合之间不进行技能的修改或添加；RoboRSI 的区别在于，它能够利用某一回合的结果来调整后续回合的技能。RoboRSI 所贡献的能力正是对当前任务的改进，因此在 LIBERO、LIBERO-PRO 和 RoboTwin 上的对比，衡量了自我改进能力相较于强大固定智能体所带来的提升。而 LIBERO-Plus 则通过在扰动实例上禁用自我改进能力来补充评估该工具库的表现。

完工前的验证。 在智能体继续执行之前，检查每个原子任务的后置条件是否与新的观测结果相符，是 RoboRSI 设计的一部分（第 3.2 节和附录 A），而非外部添加。图 6a 中提前完成比例较低，正是该设计的一个实测效果。

组成部分的证据。 第 4.5 小节分离了系统的各个组成部分：角色分离（50 个 RoboTwin 任务中 36 对比 9 对）、在相同失败情况下的 TSR 与平铺修订对比、超过 600 对匹配回合的复合技能（+7.5 点，McNemar $p < 10^{-4}$ ），骨干模型，以及从执行数据中学成的策略。

评估的范围。 主要对比包含 5,974 个评估回合：每种方法在 LIBERO 上进行 150 回合，在 LIBERO-PRO 上进行 600 回合，每个任务有五个初始状态，每种方法在 LIBERO-Plus 上进行 840 个扰动实例，RoboTwin 上每种基准方法 150 回合，RoboRSI 为 154 回合。消融实验增加了 2,770 回合，物理研究覆盖了 104 个开发演化回合。总计，RoboRSI 在四个基准上对 230 个任务进行了 1,744 回合的评估。作为参考，OpenETA 在主要的 LIBERO 评估中使用了 40 个任务，每个任务十次随机种子（共 400 回合），其自演化研究最多使用十个任务 (Chen et al., 2026b)；ASPIRE 在 LIBERO-PRO、Robosuite 和 BEHAVIOR-1K 上评估了 69 个任务 (Lu et al., 2026)；Li et al. (2026) 的记忆增强框架在 25 个任务上进行了评估，Huang et al. (2026) 在 LIBERO、LIBERO-Plus 以及五个长时域任务上进行了评估。

实施细节。 每个技能的后置条件、发布门禁、回放历史的构建以及复合技能的巩固均在该技能的描述符中指定, 并在附录 A 中总结。

4.3. 主要仿真结果

评估情景 自改进是评估能力的一部分：在 LIBERO、LIBERO-PRO 和 RoboTwin 上,RoboRSI 从初始技能库出发, 在评估任务中在线进行改进, 随着评估的进行总结失败并修订技能, 而基准方法则没有自改进机制（第 4.2 节）。LIBERO-Plus 则测试迁移能力：在 LIBERO 上获得的技能库保持固定, 并在扰动实例上不进行更新。所有方法均使用 GPT-5.6-SOL。Maestro 是成本最高的基准方法, 仅在 LIBERO 和 LIBERO-PRO 上进行评估。

Table 1 | **仿真结果**。成功指成功/有效的回合（比率）；覆盖指解决/总任务数（比率）, 每个任务在首次成功后仅计一次。LIBERO 使用 30 个任务 × 5 个回合,LIBERO-PRO 使用 120 个任务 × 5 个回合,LIBERO-Plus 使用 30 个任务的 840 个扰动实例, 而 RoboTwin 作为基准使用 50 个任务 × 3 个回合；对于 RoboRSI, 我们报告其在同一组 50 个任务上在线运行的前 154 个回合的结果。RoboRSI 在线性能优于 LIBERO、LIBERO-PRO 和 RoboTwin, 且在 LIBERO-Plus 上使用其固定的 LIBERO 库。由于计算成本较高,Maestro 仅在 LIBERO 和 LIBERO-PRO 上进行评估。

Benchmark	Method	Success	Coverage
LIBERO	CaP-X	36/150 (24.0%)	16/30 (53.3%)
	Maestro	62/150 (41.3%)	23/30 (76.7%)
	OpenETA	76/150 (50.7%)	23/30 (76.7%)
	RoboRSI	**84/150 (56.0%)**	**24/30 (80.0%)**
LIBERO-PRO	CaP-X	111/600 (18.5%)	46/120 (38.3%)
	Maestro	198/600 (33.0%)	77/120 (64.2%)
	OpenETA	231/600 (38.5%)	80/120 (66.7%)
	RoboRSI	**297/600 (49.5%)**	**102/120 (85.0%)**
LIBERO-Plus	CaP-X	130/840 (15.5%)	21/30 (70.0%)
	OpenETA	306/840 (36.4%)	27/30 (90.0%)
	RoboRSI	**354/840 (42.1%)**	**28/30 (93.3%)**
RoboTwin	CaP-X	3/150 (2.0%)	3/50 (6.0%)
	OpenETA	32/150 (21.3%)	17/50 (34.0%)
	RoboRSI	**37/154 (24.0%)**	**26/50 (52.0%)**

任务执行与覆盖。 RoboRSI 在每个基准测试中均取得了最高的成功率和覆盖度（表 1）。其最大优势体现在 LIBERO-PRO 上, 成功率比 OpenETA 高出 11.0 个百分点, 覆盖度高出 18.3 个百分点, 因此解决的任务集范围更大。在 LIBERO 上, 其空间任务（Spatial）成功率比 OpenETA 高出 14.0 个百分点, 物体任务（Object）高出 8.0 个百分点（第 4.3.1 节）。

LIBERO-PRO 优势的来源。 RoboRSI 在每个套件和每种扰动下均表现最佳（表 2）。在物体套件（+16.5）以及物体扰动（+15.3）情况下, 其优势最为显著, 这类情况需要在当前场景中重新识别目标。

Table 2 | **LIBERO-PRO 按任务套件和扰动类型划分**。成功率 (%)；每个套件包含 200 回合, 每种扰动类型每种方法包含 150 回合。粗体：列中最佳值；灰色：RoboRSI 相对于基准的提升百分点。

	Task suite			Perturbation type				
Method	Spatial	Object	Goal	Language	Object	Position	Task	All
CaP-X	24.5	13.5	17.5	21.3	16.7	17.3	18.7	18.5
Maestro	39.5	38.0	21.5	34.7	30.0	32.0	35.3	33.0
OpenETA	50.0	36.5	29.0	42.7	33.3	36.0	42.0	38.5
RoboRSI	**57.0** (+7.0)	**54.5** (+16.5)	**37.0** (+8.0)	**52.0** (+9.3)	**48.7** (+15.3)	**45.3** (+9.3)	**52.0** (+10.0)	**49.5** (+11.0)

受摄动影响的转移。 在 LIBERO-Plus 上, 冻结源库比 OpenETA 高出 5.7 分, 并且解决了 30 个任务中至少 28 个任务的至少一个扰动实例。在背景 (+13)、语言 (+12)、机器人初始状态 (+9)、噪声 (+8) 和光照 (+7) 扰动下, 性能提升最大（图 6b 报告了全部七种类型）。

基准为何失效。 在 OpenETA 和 Maestro 于 LIBERO-PRO 上的失败中, 约三分之二（66% 和 64%）以及 OpenETA 在 LIBERO-Plus 上的失败中约三分之二（66%）属于过早完成：智能体在工具报告释放后即宣布任务完成, 而模拟器仍判定任务未完成（图 6a）。对于 RoboRSI, 过早完成导致的失败占比分别为 10% 和 14%。不同方法在判断任务完成方式上存在差异：基准方法接受工具返回成功结果即视为完成, 而每个 RoboRSI 原子任务在智能体继续执行前都会根据新的观测值检查后置条件（公式 (1)）。这种完成规则的差异也体现在失败构成中,RoboRSI 剩余的失败大多发生在恢复阶段因预算耗尽而终止, 而非错误地宣称成功。在出现执行失败的回合中,RoboRSI 在 LIBERO-PRO 上仍能完成 67.0%, 在 LIBERO-Plus 上为 63.2%, 而 OpenETA 的对应数值分别为 55.0% 和 51.6%（图 6c）。

4.3.1. LIBERO 套件结果

表 3 按套件拆分了表 1 中的 LIBERO 结果；每个套件包含 10 个任务, 每种方法有 50 个回合。

4.3.2. 一日自我迭代

为了检验无需人工干预的自我迭代, 我们让 RoboRSI 在 120 个任务的 LIBERO 目录和 120 个 LIBERO-PRO 任务上自主迭代一天。在每一轮中, 当前的库在尚未解决的任务上运行, 通过验证的修订版本将在下一轮发布。图 7报告了至少被解决一次的任务累计数量。在五轮内,120 个

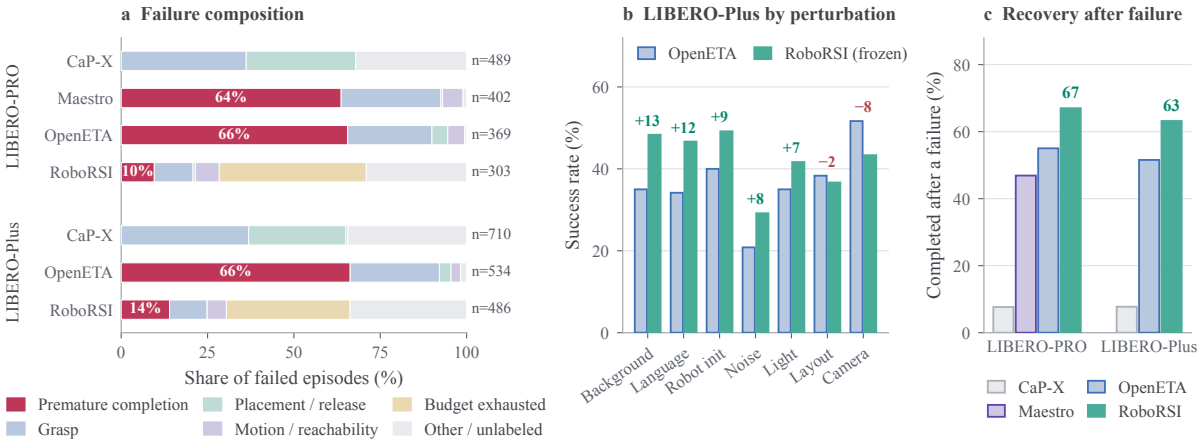


Figure 6 | **故障分析.** (a) LIBERO-PRO 和 LIBERO-Plus 上失败回合的组成；n 为失败回合的数量, 提前完成的比例已标注。(b) 按扰动类型划分的 LIBERO-Plus 成功率, 每类 120 个实例; 数值表示 RoboRSI 相对于 OpenETA 的百分点差异。(c) 在包含每个方法完整迹线且至少有一次执行失败的 209 组 LIBERO-PRO 和 258 组 LIBERO-Plus（任务, 种子）对中, 执行失败后仍能成功的回合比例（附录 C.1）。

Table 3 | **LIBERO by suite.** 每套系统的成功/有效回合数。

Method	Spatial	Object	Goal	All
CaP-X	12/50	15/50	9/50	36/150
Maestro	22/50	24/50	16/50	62/150
OpenETA	23/50	29/50	**24/50**	76/150
RoboRSI	**30/50**	**33/50**	21/50	**84/150**

LIBERO 任务的覆盖范围从 32 增加到 71,120 个 LIBERO-PRO 任务的覆盖范围从 43 增加到 80；这五轮 LIBERO 迭代共耗时 7.0 小时的有效执行时间。这些计数是跨发布的库版本累积的，与表 1中的评估结果相互分离。

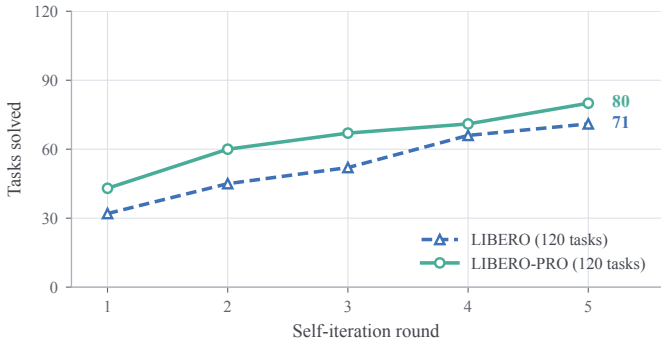


Figure 7 | **一日自迭代**。每次自迭代回合后，至少成功解决一次的任务累计数量；每个任务仅在其首次记录成功时被计数。

4.4. 案例研究

我们考察图 9中的三个回合，在这些回合中,RoboRSI 成功而基准方法失败，且均从相同的状态开始，随后遵循一个经过修改的技能，并在它原本并未开发的任务上重复使用该技能（图 8）。

在光照扰动下的终端检测。 在 LIBERO-Plus 任务中，目标是将奶油奶酪包装盒放入篮子中。OpenETA 的综合抓取与放置工具报告称物体已在目的地释放，智能体立即宣布成功。CaP-X 的放置也报告了释放，但其包含性检查仍未验证。两个回合均失败。RoboRSI 由于其源库被冻结，接收到的放置结果表明夹爪未打开。它没有直接结束，而是检查是否仍握住包装盒，重新从头部和顶部视角观察篮子，再次放置包装盒，最终成功。

放置失败后的双臂状态。 在 RoboTwin 上，两个罐子必须由分立的机械臂分别抓取并放入一个塑料箱中。CaP-X 多次尝试用左臂越过中线去够箱子，这是不可行的。OpenETA 在夹爪报告释放后停止，但未验证是否保持了容器内约束，任务失败。当 RoboRSI 首次左臂放置失败时，它继续保持两个罐子在手中，验证了两次抓取以及负载臂之间的间隙，然后依次从一侧释放罐子。

多步骤任务中的恢复 在 LIBERO-PRO 上，机器人必须打开上方的抽屉并将一个碗放入其中。CaP-X 调用了拉动抽屉的动作，而不是抽屉把手，因此被拒绝；OpenETA 多次无法接近把手，随后也无法抓取碗。RoboRSI 同样经历了拉动和放置的失败，但持续观察并采取行动，直到任务成功完成。

在所有三个回合中，基准方法因盲目信任工具的返回值而失败，而 RoboRSI 的技能能够暴露失败的结果，其智能体在继续执行前会验证后置条件，这正是图 6a 中失败组合背后的机制。

在不同任务间复用修改后的技能。 开发过程中进行的修订以共享技能的形式发布, 并应用于从未进行过开发的任务。基础技能 `grasp_object` 在单一开发任务上进行了修订：经理的补丁使每次重试都从新的观测中验证目标的身份, 并禁止重复使用已失败的抓取点, 该修订在发布前通过了门控测试。发布后, 其他任务中有 42 个成功的回合调用了这一修订后的技能。图 8 展示了其中三个回合, 分别来自目标、物体和空间套件, 这些套件在之前的冻结评估中均曾失败。由于技能通过层级结构共享, 某一任务中的修复会自动对所有调用该技能的分支可用。

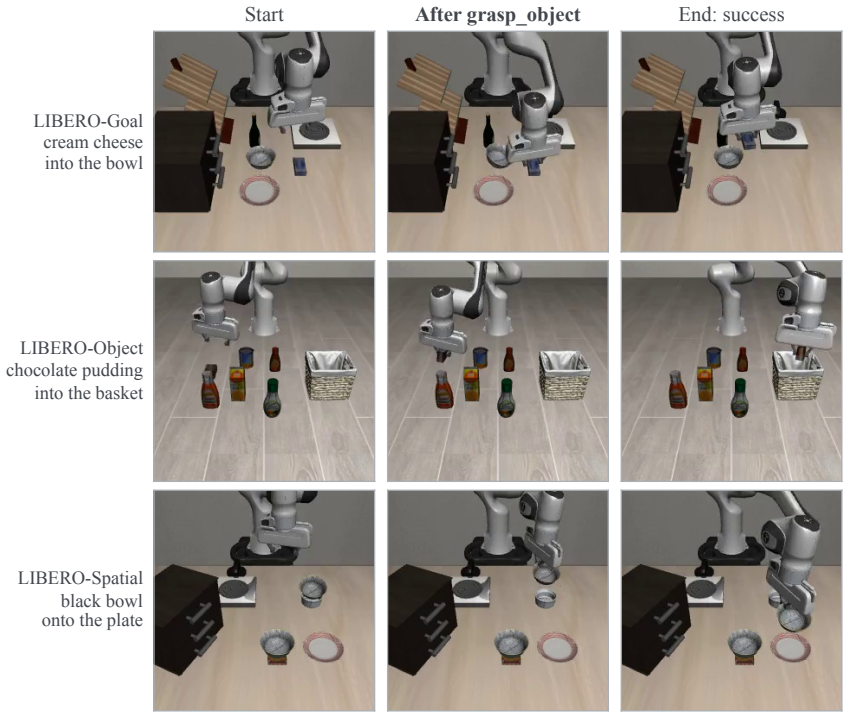


Figure 8 | **在新任务上复用修订后的技能**。在单一任务上开发的修订版 `grasp_object` 被调用到其开发集之外的三个任务中, 这些任务在之前的冻结评估中均曾失败。列显示了首次记录的帧、调用 `grasp_object` 后的状态以及最终成功状态。

4.5. 消融研究

我们分别研究角色分离、层级细化、代码合并和骨干网络选择, 并额外考察从执行数据中学习策略。骨干网络研究使用 LIBERO-PRO 和 RoboTwin 任务面板, 每个任务进行三个评估回合, 每种骨干网络共进行 360 和 150 个回合。

4.5.1. 角色分离

我们对比了在 50 个 RoboTwin 任务上使用单智能体与分离角色的开发情况（表 4）。在单智能体条件下, 一个工程师负责规划、执行并评估自身尝试的结果。在多智能体条件下, 一个规划者制定计划, 工程师执行计划, 而审查者则根据观测结果和工具迹来诊断每次执行的结果。单智能体解决了 50 个任务中的 9 个, 而多智能体系统解决了 36 个。将诊断与执行分离, 使得每次失败都能在下一次尝试前获得独立的审查, 而不是让执行智能体自行判断其结果。

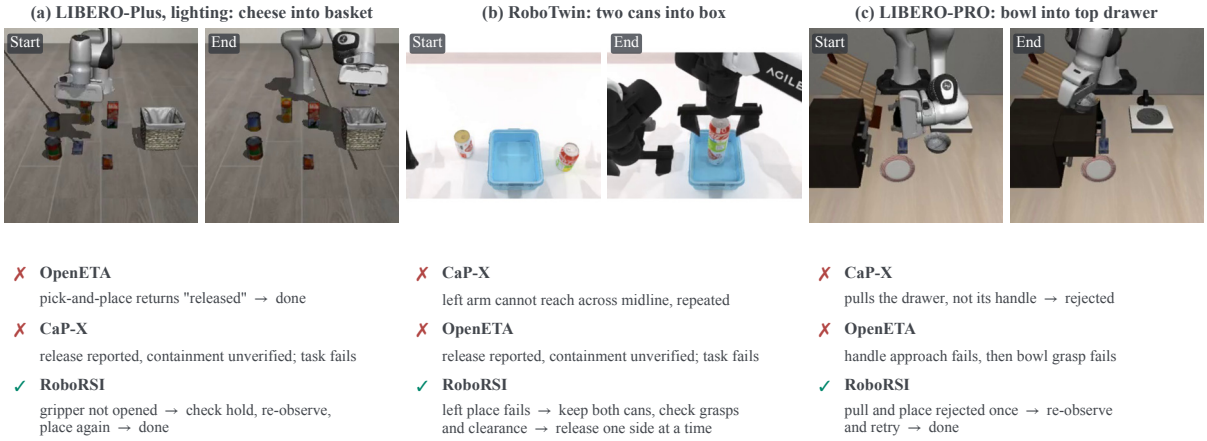


Figure 9 | **不同方法下初始状态相同的仿真案例**。图像展示了 RoboRSI 回合的第一帧和最后一帧；文本总结了每种方法的关键工具调用。

Table 4 | **RoboTwin 上的单智能体与多智能体开发**。在完整的在线运行过程中至少解决一次的任务，共 50 个；多智能体条件是表 1 中的 RoboTwin 运行，其中报告了前 154 个回合。

Condition	Roles	Tasks solved
Single agent	Engineer	9/50 (18.0%)
Multi-agent	Planner, Engineer, Reviewer	**36/50 (72.0%)**

4.5.2. 自顶向下技能优化

TSR 决定了系统改进的位置。因此，我们比较 TSR 与平铺式修订在将观测到的失败转化为已验证、已发布的改进方面的表现，而非任务成功率。两种条件均从相同的代码和记忆状态开始，并接收相同的一次 `libero_pick_place` 在 `libero_goal/8` 上的失败公开迹。它们均可修改原子技能和基础技能，并使用相同的规划器、工程师、评审员和经理评审流程。只有当基础功能检查通过且在两个开发种子上原子任务均成功完成时，候选方案才会被发布。在 TSR 中，评审员首先从原子失败构建责任树，并选择需要修订的层；而在平铺式修订中，它直接从失败提出修订建议。

TSR 将失败归因于两个结点：原子计划在经过验证的释放后，将拾取时设置的“不重新抓取”标志视为永久有效，而基础持有检查在运输过程中将固定的拾取贴片视为不变。其候选修改仅针对这两个结点，在两个种子上均成功，并已发布（表 5）。平坦候选方案还重写了放置策略，在一个种子上成功；在另一个种子上，智能体声明成功时物体已离开目标。进一步的平坦候选方案在经理审查后仅限于原子计划，再次在一个种子上成功，因此未发布任何平坦候选方案。

4.5.3. 代码合并

我们对比了智能体在具备复合技能（代码开启）与未提供复合技能（代码关闭）情况下的执行表现，测试集为从 LIBERO-Plus 中独立选取的 120 个任务，与表 1 中的任务集不同，因此其绝对成功率无法与主结果直接比较。两种条件下使用的基础工具、GPT-5.6-SOL 主干模型、120 个任务

Table 5 | **TSR 与同一失败任务的平铺修订**。文件和行数的变化是相对于共同起始代码而言的；原子成功性基于两个开发种子。

Condition	Files	Changed lines	Atomic success	Released
TSR	3	50	**2/2**	yes
Flat, candidate 1	4	104	1/2	no
Flat, candidate 2	1	63	1/2	no

以及五个随机种子均相同，因此每个代码开启的回合都有一个对应的代码关闭对照回合。图 10 报告了这 600 对匹配回合的成功率，以及在另一组独立的 118 个任务样本上的规范化的中位数成本。

代码合并将成功率从 21.5%（129/600）提升至 29.0%（174/600），配对提升 7.5 个百分点（95% 置信区间 $[3.7, 11.5]$；McNemar $p < 10^{-4}$），且至少解决一次的任务数从 58 增加至 120 中的 71。该提升分布在多个回合中：86 组匹配对仅在启用代码合并时成功，而仅在关闭代码合并时成功的有 41 组。中位数的 token 消耗量、模型调用次数和实际运行时间分别减少了 29.4%、27.2% 和 17.0%。

成功收益源于长工具序列中错误的产生方式。在没有代码的情况下，智能体在每个回合中都需要重新构建序列，每一步都是再次选择错误工具、传递不一致参数或跳过步骤间检查的机会；在包含大量步骤的序列中，这些错误会累积。复合技能固定了调用顺序和步骤间的检查，因此智能体只需决定调用哪个技能以及使用哪些参数。第 3.2 节中的物理 Round 25 失败展示了相同的效果：由模型编写的释放序列遗漏了代码生成序列中包含的步骤。成本降低也源于同一原因，因为一次技能调用替代了多次模型决策。由于模拟器中执行的动作保持不变，仅移除了动作之间的推理，因此实际耗时的下降幅度小于 token 数量和模型调用次数的减少。当复合技能报告失败时，智能体仍会响应执行反馈，并返回到逐步推理。

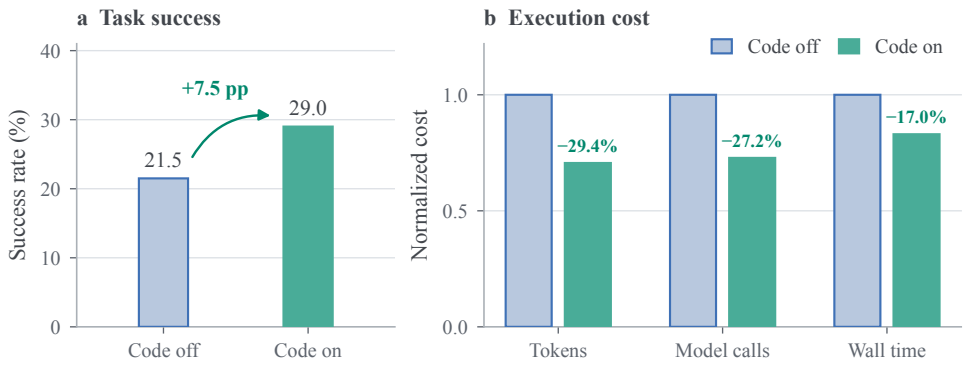


Figure 10 | **代码合并的效果**。(a) 每种条件下 600 个匹配回合的任务成功率。(b) 在 118 个任务效率样本上的中位数 token、模型调用和时间成本，以关闭代码（Code off）为基准进行规范化。

4.5.4. 骨干模型

我们在推理角色上对 GPT-5.6-SOL、GPT-5.5 和 GPT-5.4 进行比较, 三者具有相同的初始技能、任务分布和交互预算（表 6）。GPT-5.5 在 LIBERO-PRO 任务中取得了最高的成功率, 为 48.6%, 并在物体与目标套件中领先, 而 GPT-5.6-SOL 在 RoboTwin 套件中表现最佳。模型调用次数差异极小, 每个 LIBERO-PRO 回合的中位数在 115 到 124 之间, 因此更强的主干网络并非通过更长时间推理取胜。差异在于回合耗尽预算的频率：GPT-5.4 在 101 个回合中耗尽预算, 而 GPT-5.5 仅在 80 个回合中耗尽, 说明更强的主干网络能更频繁地在预算范围内达成验证结果。由于不同基准下的秩次发生变化, 而任务结构与技能库保持不变, 因此主干选择与基准存在交互作用, 而非在所有基准中始终偏好某一模型。

Table 6 | **主干网络比较**。该研究使用了一个独立的 120 个 LIBERO-PRO 任务面板 ×,3 次随机种子独立运行, 与表 1无关, 因此仅比较不同主干网络之间的相对差异。每个主干网络在 360 个回合中的 LIBERO-PRO 成功率, 总体及按任务套件划分；RoboTwin 在 150 个回合中的成功率。模型调用次数为每个 LIBERO-PRO 回合的中位数；预算耗尽次数统计了在交互上限结束的 LIBERO-PRO 回合。粗体：该列最佳值。

	LIBERO-PRO success (%)				RoboTwin	LIBERO-PRO episodes	
Backbone	All	Spatial	Object	Goal	success	Model calls	Budget exhausted
GPT-5.6-SOL	38.3	38.3	48.3	28.3	**24/150**	115	79
GPT-5.5	**48.6**	53.3	**55.0**	**37.5**	19/150	122	80
GPT-5.4	40.3	**55.8**	39.2	25.8	21/150	124	101

4.5.5. 从执行数据中学习策略

成功的 RoboRSI 执行为训练紧凑的视觉-运动策略提供了示范, 这些策略随后可在层级结构中作为技能使用。我们基于 10 次成功的 `click_bell` 和 7 次成功的 `click_alarmclock` 执行来训练 ACT 策略 (Zhao et al., 2023), 并在相同的数据、架构和 10,000 次训练步骤下比较两种动作表示方法。采用绝对关节目标时, 在任何评估布局中均未能解决任一任务。将关节目标相对于每个动作块开始时的机械臂配置进行表达后, 铃铛任务在训练布局上的成功率提升至 4/10, 闹钟任务在训练布局上的成功率为 6/7, 在未提供训练数据的收集布局上成功率为 2/3（图 11）。

由于示范数量极少, 绝对目标要求策略在每一步都回归完整的机械臂配置, 而微小的回归误差会将末端执行器移出这些按压任务所需的微小接触区域。而块相对目标仅编码了机械臂从当前配置移动的方式, 这使得示范对其约束更为紧密：在训练观测数据上, 关节指令误差降低了 57%, 从 0.0086 降至 0.0037 弧度。

附录 C.3 提供了训练细节。

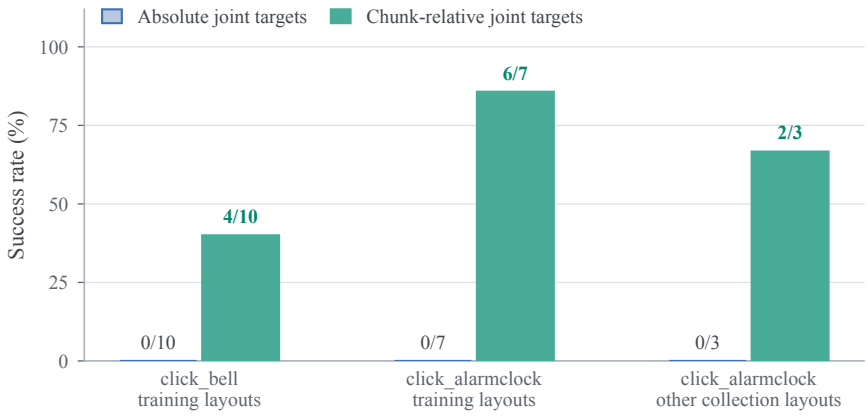


Figure 11 | **在 RoboRSI 执行中训练的 ACT 策略**。在相同的示范、架构和训练预算下, 成功实现绝对和分块相对关节目标。

5. 结论与未来工作

RoboRSI 通过自顶向下技能精炼与多智能体协同, 组织执行、诊断、局部修正和可复用能力, 以实现复杂现实环境中稳定、高效且可复用的机器人自演化。未来工作将把仿真中获取的经验迁移到物理机器人, 并在机器人及平台间共享技能。

References

J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Binkowski, R. Barreira, O. Vinyals, A. Zisserman, and K. Simonyan. Flamingo: a visual language model for few-shot learning. In Advances in Neural Information Processing Systems (NeurIPS), 2022. URL https://arxiv.org/abs/2204.14198.

K. Black, N. Brown, D. Driess, et al. π_0: A vision-language-action flow model for general robot control. In Robotics: Science and Systems (RSS), 2025. URL https://arxiv.org/abs/2410.24164.

A. Brohan, N. Brown, J. Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In Conference on Robot Learning (CoRL), 2023. URL https://arxiv.org/abs/2307.15818.

B. Chen, B. Zhang, F. Tang, Z. Lu, Y. Du, T. Chen, W. Lu, J. Xiao, Y. Zhuang, and Y. Shen. Reflect, revise, reuse: Training-free skill evolution for GUI agents. arXiv preprint arXiv:2609.17653, 2026a. URL https://arxiv.org/abs/2609.17653.

Y. Chen, Z. Huai, S. Li, Y. Wang, H. Zhang, Y. Zhang, H. Chen, J. Gong, Y.-G. Jiang, and X. Qiu. ETA: A new agentic paradigm for embodied tasks. arXiv preprint arXiv:2608.03924, 2026b. URL https://arxiv.org/abs/2608.03924.

D. Cheng, T. Yi, Y. Fang, X. Zhang, F. Feng, Y. Li, G. Zhuang, R. Wang, S. Yang, W. Song, W. Xue, M. Wu, J. Gui, J. Wang, and T. Wu. In-context robot learning with VLM agents. arXiv preprint arXiv:2609.19138, 2026. URL https://arxiv.org/abs/2609.19138.

D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence. PaLM-E: An embodied multimodal language model. In International Conference on Machine Learning (ICML), 2023. URL https://arxiv.org/abs/2303.03378.

J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In International Conference on Learning Representations (ICLR), 2025. URL https://arxiv.org/abs/2410.00371.

Y. Fan, Z. Di, Z. Wen, Y. Yang, M. Cheng, Q. Dai, B. Liu, K. Qiu, Y. Dong, J. Li, and C. Luo. RESOURCE2SKILL: Distilling executable agent skills from human-created multimodal resources. arXiv preprint arXiv:2606.29538, 2026. URL https://arxiv.org/abs/2606.29538.

L. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, D. Niu, F.-F. Li, G. Shi, J. Wu, S. Sastry, Y. Zhu, K. Goldberg, and L. J. Fan. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. arXiv preprint arXiv:2603.22435, 2026. URL https://arxiv.org/abs/2603.22435.

Gemini Robotics Team et al. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer. arXiv preprint arXiv:2510.03342, 2025. URL https://arxiv.org/abs/2510.03342.

Gemini Team et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530, 2024. URL https://arxiv.org/abs/2403.05530.

Y. Guo, Y.-J. Wang, L. Zha, and J. Chen. DoReMi: Grounding language model by detecting and recovering from plan-execution misalignment. In IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2024. URL https://arxiv.org/abs/2307.00329.

J. Huang, Y. Hu, Z. Li, R. Qi, Y. Xiao, Z. Zhang, M. Coates, T. Cao, and Y. Zhang. RoboHarness: Memory-driven orchestration of heterogeneous robot policies for long-horizon planning. arXiv preprint arXiv:2607.18060, 2026. URL https://arxiv.org/abs/2607.18060.

W. Huang, P. Abbeel, D. Pathak, and I. Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In International Conference on Machine Learning (ICML), 2022a. URL https://arxiv.org/abs/2201.07207.

W. Huang, F. Xia, T. Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. In Conference on Robot Learning, 2022b. URL https://arxiv.org/abs/2207.05608.

W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and F.-F. Li. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In Conference on Robot Learning (CoRL), 2023. URL https://arxiv.org/abs/2307.05973.

W. Huang, C. Wang, Y. Li, R. Zhang, and F.-F. Li. ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. In Conference on Robot Learning (CoRL), 2024. URL https://arxiv.org/abs/2409.01652.

R. Ju, X. Wang, X. Ding, Y. Yang, H. Wu, S. Jiang, Q. Zhang, H. Wen, X. Li, W. Wang, K. Li, Y. Liu, H. Dai, W. Wang, and T. Cao. EmbodiSkill: Skill-aware reflection for self-evolving embodied agents. arXiv preprint arXiv:2605.10332, 2026. URL https://arxiv.org/abs/2605.10332.

M. J. Kim, K. Pertsch, S. Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In Conference on Robot Learning (CoRL), 2024. URL https://arxiv.org/abs/2406.09246.

Z. Li, Z. Li, K. Zhou, and J. Gu. RoboHarness: A memory-augmented policy harness for vision-language-action model robustness via in-context adaptation. In IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2026. URL https://arxiv.org/abs/2603.24060.

J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In IEEE International Conference on Robotics and Automation (ICRA), 2023. URL https://arxiv.org/abs/2209.07753.

H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. In Advances in Neural Information Processing Systems (NeurIPS), 2023a. URL https://arxiv.org/abs/2304.08485.

Z. Liu, A. Bahety, and S. Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In Conference on Robot Learning (CoRL), 2023b. URL https://arxiv.org/abs/2306.15724.

R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, M. Chowdhury, Y. Zhu, L. J. Fan, and G. Wang. ASPIRE: Agentic /skills discovery for robotics. arXiv preprint arXiv:2607.00272, 2026. URL https://arxiv.org/abs/2607.00272.

A. Madaan, N. Tandon, P. Gupta, et al. Self-refine: Iterative refinement with self-feedback. In Advances in Neural Information Processing Systems, 2023. URL https://arxiv.org/abs/2303.17651.

Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In Conference on Robot Learning (CoRL), 2025. URL https://arxiv.org/abs/2504.16054.

J. Shi, R. Yang, K. Chao, S. B. Wan, Y. Shao, J. Lei, J. Qian, L. Le, P. Chaudhari, K. Daniilidis, C. Wen, and D. Jayaraman. Maestro: Orchestrating robotics modules with vision-language models for zero-shot generalist robots. arXiv preprint arXiv:2511.00917, 2025. URL https://arxiv.org/abs/2511.00917.

N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, 2023. URL https://arxiv.org/abs/2303.11366.

I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg. ProgPrompt: Generating situated robot task plans using large language models.

In IEEE International Conference on Robotics and Automation (ICRA), 2023. URL https://arxiv.org/abs/2209.11302.

M. Sun, J. Wei, Q. He, D. Wang, C. Lu, and J. Sun. Executable analytic concepts as the missing link between VLM insight and precise manipulation. arXiv preprint arXiv:2510.07975, 2025. URL https://arxiv.org/abs/2510.07975.

G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. Transactions on Machine Learning Research, 2024. URL https://arxiv.org/abs/2305.16291.

W. Xiao, J. Xie, T. Zhang, H. Lin, L. Fu, H. Xue, J. Lu, Y. Yang, C. Dai, Z. Wang, J. Wu, G. Wang, S. S. Sastry, K. Goldberg, L. Fan, Y. Zhu, and G. Shi. ENPIRE: Agentic robot policy self-improvement in the real world. In Conference on Robot Learning (CoRL), 2026. URL https://arxiv.org/abs/2606.19980.

X. Xu, J. Liu, Z. Qiao, J. Lu, X. Zhang, Y. Gu, F. Ning, and Y. Shi. OmniHarness: Harnessing generalizable visual generation via symbolic policy learning. arXiv preprint arXiv:2609.16057, 2026. URL https://arxiv.org/abs/2609.16057.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In International Conference on Learning Representations, 2023. URL https://arxiv.org/abs/2210.03629.

H. Ye, Y. Xiao, C. Lu, and P. Cai. UniDomain: Pretraining a unified PDDL domain from real-world demonstrations for generalizable robot task planning. arXiv preprint arXiv:2507.21545, 2025. URL https://arxiv.org/abs/2507.21545.

H. Ye, Y. Xiao, C. Lu, and P. Cai. UniPlan: Vision-language task planning for mobile manipulation with unified PDDL formulation. arXiv preprint arXiv:2602.08537, 2026. URL https://arxiv.org/abs/2602.08537.

T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In Robotics: Science and Systems (RSS), 2023. URL https://arxiv.org/abs/2304.13705.

E. Zhou, Q. Su, C. Chi, Z. Zhang, Z. Wang, T. Huang, L. Sheng, and H. Wang. Code-as-Monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025. URL https://arxiv.org/abs/2412.04455.

J. Zhou, Q. Zhang, G. Xu, C. Fan, Y. Zhao, R. Wang, Y. Luo, S. Yang, X. Zhu, Y. Shen, J. Liang, and Y. Xu. Zero-WAM: In-context world-action modeling from human videos for

open-ended task generalization. arXiv preprint arXiv:2608.26103, 2026. URL https://arxiv.org/abs/2608.26103.

A. TSR 配方与分析

任务-技能图谱与作用域补丁。 在迭代 t 时, 释放的层次结构由 $G_t = (V_t, E_t, \{c_v\}_{v \in V_t})$ 表示, 其中 V_t 包含任务和技能结点,E_t 包含可能的调用边,c_v 为结点的实现或指引。执行记录 τ_t 包括目标、观测结果和工具迹。诊断确定受影响的范围 $S_t \subseteq V_t$; $G_t|_{S_t}$ 包含其结点和内部调用, 而 ∂S_t 包含与其与未改变层次结构相连的接口。

修订过程 $\mathcal{R}$ 提出一个图和代码补丁 Δ_t , 其中包含 $\mathrm{supp}(\Delta_t) \subseteq S_t$ 。支持部分识别出其实现、指导或传出调用发生变化的现有结点。可在该范围内附加新的辅助结点。操作符 $\oplus$ 同时应用代码编辑和分支扩展。如果诊断结果涉及 S_t 之外的调用者或接口, 则在下一次提议前扩大作用范围。

历史与验证。 公式 2 中的案例集合为 $\mathcal{H}_t = \mathcal{D}_t^- \cup \mathcal{D}_t^+(S_t)$, 包含近期导致需要编辑的失败案例以及受影响调用者中选定的成功案例。通过审查和功能测试, 判断 $\mathcal{V}$ 是否返回提议的补丁或空补丁。应用空补丁将使已发布的图结构保持不变; 被接受的修改将在后续执行中可用。当仅有一个实现发生变化时, 共享技能仍需检查相关调用路径。下一次迭代使用当前已发布版本的反馈信息。

依赖性局部效应。 公式 2 将编辑的作用范围与其可能影响的范围分离开来。设 r_q 为任务 q 的入口结点, 且 $\mathrm{Reach}_{G_t}(r_q)$ 包含该结点以及通过可能调用可达的所有结点。受影响的任务集合为 $\mathcal{A}_t = \{q : \mathrm{Reach}_{G_t}(r_q) \cap S_t \neq \emptyset\}$。这使用的是可能的依赖关系, 而不仅仅是某次滚动中观察到的调用: 一个共享的基础技能可能在不同观测下影响多个任务。

考虑一个固定的任务分布 $p(q)$, 令 $J_q(G)$ 为在图 G 下任务 q 的期望失败率。假设该图捕获了所有行为依赖关系, 补丁对其作用范围外的接口和实现保持不变, 且评估使用相同的重置和外部随机性分布。特别地, 编辑不得改变未追踪的共享状态、全局提示或工具配置。在这些条件下,$\mathcal{A}_t$ 以外的任务不会遇到已更改的实现。它们的执行分布保持不变, 因此在总体差异中相互抵消:

$$\begin{aligned} J(G_{t+1}) - J(G_t) &= \sum_{q \in \mathcal{A}_t} p(q)\big[J_q(G_{t+1}) - J_q(G_t)\big], \\ |J(G_{t+1}) - J(G_t)| &\le \sum_{q \in \mathcal{A}_t} p(q), \qquad J(G) = \sum_q p(q) J_q(G). \end{aligned} \tag{3}$$

该界限成立的原因是每个失败率均位于 $[0,1]$ 内。其大小与受影响任务的概率质量成比例, 并同时覆盖改进和回归情况。特定任务的分支可能只影响少量任务, 而广泛共享的基础技能则可能影响大量任务。这促使我们根据依赖覆盖度而非修改函数的数量来选择回归案例。如果某次修订改变了共享状态或接口, 则必须在分析前扩大其依赖范围。开发案例指导修订过程, 而保留的任务用于衡量泛化能力。

代币会计 建设成本包括技能生成、诊断、审查和验证。执行成本包括执行智能体内部的所有模型调用, 包括基于模型的感知、监控和回退; 单独计量的规划和审查不包含在此执行小计中。未

计量的调用将单独报告。比较时需要匹配的任务分布, 并且必须包含失败的执行, 因为较短的失败尝试可能消耗更少的 token 但并未提高效率。任务成功率、运行时间以及回退到在线推理的频率为解释执行 token 减少提供了上下文。

执行与结果接口。 智能体通过计划、结构化的技能输入与输出以及执行记录进行通信。物理部署采用人工指导和安全监督; 仿真可以自动完成重置和结果检查。规划者、工程师和评审者接收观测信息和工具反馈, 而隐藏的仿真器状态和成功条件的内部机制则不在其上下文范围内。任务执行后, 仿真器会单独对任务进行评分。

操作细节。 每个技能都有一个描述符, 声明其参数、带类型的返回字段、适用条件以及验证工具(仿真任务、种子、参数和任何设置技能)。返回字段报告预期效果是否已验证; 例如, 放置技能返回目标是否到达、夹爪是否打开以及释放是否已验证, 且仅在达到和抓取检查通过后才打开夹爪。原子任务只有在其后置条件在最后一次动作后的全新观测中成立时才算完成。候选修订版本会在库的隔离副本中暂存, 并由经理审查。如果修改了基础技能, 则该技能的工具将在仿真器中运行; 随后, 完整的原子任务将在其开发种子上重新执行。发布需要在至少两个声明的种子上仿真成功, 试验后恢复未修改的源代码, 并获得经理批准且无待处理变更; 失败的候选版本无法以相同形式重新提交。在仿真中,$\mathcal{H}_t$ 包括这些工具和开发运行; 在机器人上, 它包括最近四次符合条件的运行, 离线重放 (附录 D.4)。当一个任务至少有三次成功执行, 且其工具序列共享至少 60% 的最长公共子序列, 并具有至少三个步骤的共同骨架时, 才会提出复合技能。生成的技能会重放骨架, 并在运行时从感知中重新计算每个参数, 从不存储回合中的坐标, 且其审查和验证流程与其他任何修订版本相同。

B. 物理评估情景 & 评估流程

平台与观测界面。 物理实验使用两个机器人平台。移动清理案例研究采用 WheelSingleArm M1, 该平台包含 Athena Pro Max 移动底盘、RealMan 机械臂和 Zhiyuan 夹爪。机载摄像头和底盘安装的摄像头提供 RGB-D 观测数据。智能体可访问机械臂关节状态、末端执行器状态、夹爪状态、底盘位姿以及导航与对接状态。实验前已进行相机到机械臂的校准。所有移动开发运行均使用相同的预先构建的导航地图。

双臂配置包含两台 AgileX PiPER 机器人臂, 配备标准的 100 毫米 PiPER 夹爪。一个面向工作区的 Intel RealSense D435i 相机, 以及安装在每只手腕上的一个 Intel RealSense D405 相机, 提供 RGB-D 观测数据。智能体可获取全部三个相机流的数据, 以及两个机械臂的关节状态、末端执行器位姿和夹爪状态。实验前已完成相机与机械臂之间的校准。

移动任务分发与开发协议。 移动清理开发记录共包含 104 次运行。每次运行均从一个共同的停靠姿态开始。执行前, 现场人员会将场景恢复至一个统一的基础布局, 同时改变物体的数量、身份、位置和朝向, 以及其目标容器的位置。任务清单指定了该次运行中的物体和目标位置, 并在

执行过程中保持不变。机器人需将指定物体放置到指定容器中, 并返回至停靠姿态。

RoboRSI 可在不同运行之间调整其技能, 而在每次运行中技能版本保持不变。仅在需要时才进行修订, 而非每次运行后都进行。候选修订方案在后续运行中使用前, 会与保留的、经过审查的执行记录进行比对；为进行此项比对, 不会实际重复执行早期任务。因此, 这 104 次运行追踪的是一个随不同任务配置不断演进的系统的发展过程, 而非对同一任务反复试验一个固定策略。

结果标注与图表构建。 当现场人员观察到物体进入其指定的容器时, 即计为一次放置。对于图 4 中选定的每一次运行, 每次计数的放置均通过这种方式确认。该图展示了十个事后选择的运行实例, 以代表早期的部分执行、多物体放置、开发过程中的挫折以及后期完成的任务。这些运行实例的选择是在检查开发记录之后作出的, 而非根据预先规定的采样规则。

所绘制的数值是单次运行中前四项处理任务的确认放置数量。四项作为常见的显示尺度；如果某次运行的清单中少于四项, 则不视为四项目试验。标记的出现表示经审核的任务结果。由于各次运行中的物体组合和重置配置各不相同, 因此所绘制的数值描述的是开发过程中的放置进展, 而非固定任务的成功率。

移动执行记录。 记录的示范包含两个清理场景, 分别有五个和四个目标物体（图 5）。现场人员确认, 每个物体均被放置在指定容器中。场景之间, 操作员重新排列了物体和容器；技能代码、提示和策略配置保持不变。两次清理执行均无需人工接管或修正。

一次独立的物理评估尝试展示了放置失败的情况：机器人抓取了浴球, 但将其释放到了目标桶之外。执行迹线揭示了释放检查与所需最终状态之间的不匹配, 从而引导对放置几何形状的后续修正。该修正通过保留的观测数据进行了验证, 但在匹配的修复后物理试验中未进行评估。此故障案例单独报告, 不同于两场景演示和 104 次运行的开发迹线。

C. 补充实验

本附录报告了失败分类、探索性开发及小规模策略学习。

C.1. 故障分类

每个失败的回合根据其公开工具结果和模拟器的终端结果, 由确定性规则分配一个类别。诸如空抓取、物体掉落、释放被拒绝或目标不可达等显式字段, 决定了抓取、放置/释放以及运动/可达性失败；对于完整的迹, 使用最后一个显式失败步骤。当智能体声明成功而模拟器报告失败时, 该回合为过早完成；在交互次数上限结束的回合则为预算耗尽。其迹不支持上述任何规则的回合为未标记。各类别在每个评估面板中分别统计；分析覆盖了 5,008 个失败回合。在失败后仍完成（图 6c）的计算基于所有方法均有完整迹且至少经历一次执行失败的（任务, 种子）对,LIBERO-PRO 上有 209 对,LIBERO-Plus 上有 258 对；其值为这些回合中模拟器仍成功结束的比例。

C.2. 探索性开发

早期的 LIBERO-Plus 运行还允许在目标基准上进行能力更新。它们的配置支持多次尝试和目标端的更新, 而主 LIBERO-Plus 传输面板则保持源端能力固定。

C.3. 从执行数据中学成策略

示范是固定收集布局上成功的 RoboRSI 执行：`click_bell` 有十个, 生成了 2,161 个训练样本,`click_alarmclock` 有七个。两种动作表示均采用相同的 ACT 架构, 使用从零开始训练的残差网络（ResNet-18）视觉主干,Transformer 宽度为 256, 动作块大小为 100 步, 每次推理执行 25 步,KL 权重为 1, 训练步数为 10,000 步。绝对版本直接预测关节位置命令。块相对版本则预测相对于每个块起始时刻机械臂配置的关节位置, 夹爪和速度输出保持不变。最终检查点在独立的推理服务器上运行, 无需在线智能体协助, 每回合控制周期预算为 2,000 次。评估涵盖 `click_bell` 的十个训练布局以及 `click_alarmclock` 的十个收集布局, 其中七个布局提供了训练示范。

D. 案例研究：技能代码与故障驱动的修订

我们考察了在同一开发任务和初始状态下的执行失败与本地技能修订。随后的独立案例记录了技能发布以及由评审驱动的代码修订。

D.1. 执行修复：消除过早的目标排斥

此案例遵循了在 `click_bell` 上对 `guarded_compact_feature_tap` 的先前修订版本。两个候选方案均由 GPT-5.6-SOL Manager 调用进行评审, 并在相同的三个声明的初始状态、调用参数和独立模拟器成功准则下进行评估。每个版本在每种初始状态下运行一次。原始候选方案在三次试验中成功了一次（1/3）；修订后的候选方案在全部三次试验中均成功（3/3）, 且无崩溃情况。其满足了预先声明的至少两次成功的标准, 随后被发布。这六次运行构成了以下开发对比。

故障与责任结点。 在第一次失败的试验中, 原始技能在 320× 240 的图像中观察到一个铃铛的边界框 [249, 185, 306, 240]。其下边缘超过了硬编码的阈值 237, 因此目标关联检查在发出点击之前拒绝了该物体。任务因此失败。负责的代码位于技能的关联检查部分, 位于 `get_object_bbox` 之后；铃铛按压的目标和工具接口无需更改。原始的大小和边界检查如下：

```
if width < 8 or height < 8 or width > 100 or height > 100:
    return fail("object association is not within fixed compactness bounds")
if x0 < 2 or y0 < 2 or x1 > 317 or y1 > 237:
    return fail("object association touches the fixed head-camera border guard")
```

本地修订与实际执行。 在收到失败运行的公开工具迹后, 管理器生成了一次修订, 该修订允许对紧凑对象进行有界下边缘截断。以下修订检查保留了大小限制并拒绝过度截断；框内特征、深度和臂状态检查仍保留在技能的其他位置, 此次修订还增加了运动终点检查。

```
if width < 8 or height < 8 or width > 100 or height > 100:
    return fail("object association is outside fixed compactness bounds")
if x0 < 2 or y0 < 2 or x1 > 318 or y1 > 240:
    return fail("object association violates fixed image guards")
if y1 == 240 and height > 70:
    return fail("lower-edge truncation is too extensive")
```

在从相同初始状态进行的修订后运行中, 观察到的方框是 $[249, 185, 306, 239]$, 定位的特征是 $(257, 195)$。该技能接受了目标, 调用 `move_to_pixel` 并设置 `action=tap`, 并收到了悬停–靠近–按下–撤退的执行报告。独立模拟器的判定为成功。图 12 显示了相应的观测结果。在相同的记录后修订观测数据上重放旧代码时, 仍然因 $239 > 237$ 而拒绝该方框; 而修订后的代码重播了全部 22 次记录的公开调用, 包括点击操作。重播使用存储的工具输出, 对两种实现方案在相同观测下的表现进行对比。

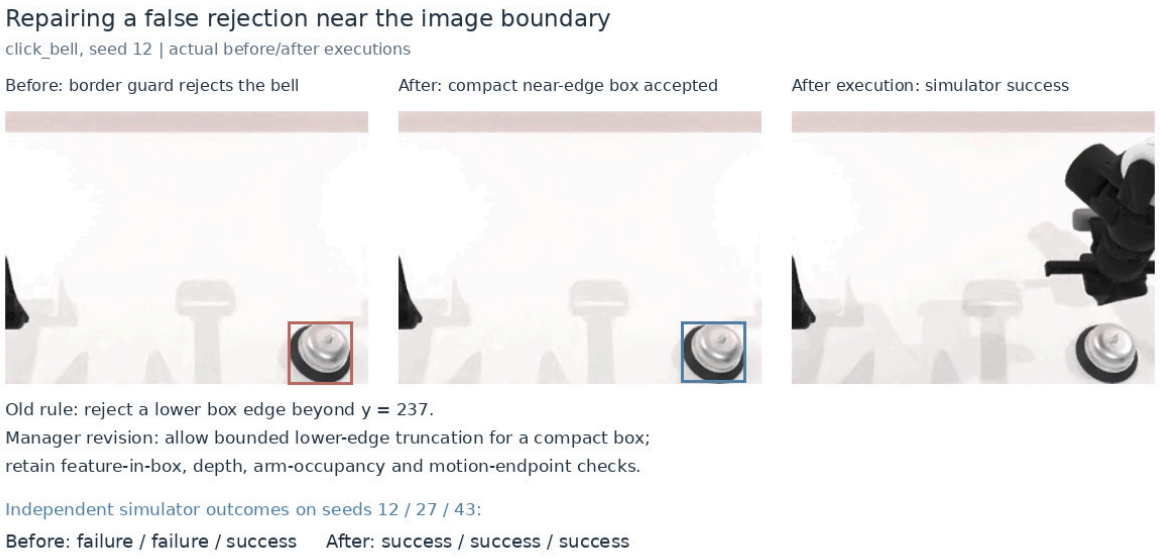


Figure 12 | 局部技能修订使得在按铃情境中能够执行。左图: 原始的目标关联检查在动作前拒绝了靠近边界的方框; 中图: 修订后的检查接受了紧凑的目标; 右图: 成功修订运行后的执行后观察结果。方框标记检测到的目标; 模拟器提供任务结果。

评估情景。 主管向经理提供了完整的工具参数、输出结果以及失败的迹, 并组织了两次验证运行。经理修改了代码, 随后进行了独立审查。该补丁更新了边界检查和恢复逻辑; 在第二次试验中, 两个版本均尝试执行点击操作。原生工具、调度程序和测试框架代码保持不变。各次运行使用了不同的 GPU 及略有差异的初始图像, 第一次试验中盒子边界的像素存在一像素差异。进程限制分别为 300 秒和 900 秒, 实际持续时间约为 231 秒和 297 秒。

D.2. 生成复合技能：验证、发表和调用

一个稍后出现的、独立的 exact_waypoint_feature_tap 技能提供了可执行能力累积的明确示例。由 GPT-5.6-SOL 管理器调用生成了修订后的代码, 随后通过另一个独立的管理器调用进行了审查。该描述符暴露了四个输入：机械臂、物体描述、功能特征描述和相机。实现过程获取一张新图像, 定位物体及其功能点, 恢复深度信息, 检查运动端点, 执行点击操作, 并检查退避动作和终止状态。其结构化输出描述了执行情况、恢复情况以及失败原因。模拟器评估任务完成情况。

以下源代码片段展示了感知后的固定执行顺序。此处 xyz 为当前观测到的特征位置,u,v 为其图像坐标,screen_tcp 验证候选终点。该片段省略了输入验证、感知及后续终端检查。代码保留了执行流程, 而对象关联和动作目标在每次调用时重新计算。

```
top_down = [0.5, -0.5, 0.5, 0.5]
approach_tcp = [xyz[0], xyz[1], xyz[2] + 0.10] + top_down
press_tcp = [xyz[0], xyz[1], xyz[2] - 0.005] + top_down

if screen_tcp(approach_tcp) is None:
    return fail("exact tap approach/retreat endpoint is unsafe or unknown")
if screen_tcp(press_tcp) is None:
    return fail("exact tap press endpoint is unsafe or unknown")

tap_result, _ = _dispatch_tool(
    state,
    "move_to_pixel",
    {
        "arm": arm,
        "u": u,
        "v": v,
        "action": "tap",
        "height_above_m": 0.0,
        "camera": "head_camera",
    },
)
tapped = tap_result.get("ok") is True
```

验证与发表。 预设的开发检查要求在至少三个 click_bell 试验中, 独立模拟器成功至少两次。其中三个试验中有两个成功, 且没有运行崩溃。2/3 的结果通过了该检查, 随后经过审查的实现被发布为一个版本化的技能。此后续技能使用其自身的三试验开发面板。

图 13 显示了成功开发试验的观测结果。

D.3. 代码前后对比：由评审驱动的终端检查

早期候选方案在经理评审阶段被拒绝。其恢复表达式存在短路情况：当运动或姿态检查失败时, 后续的占据和清空查询可以跳过。此外, 在确认占据状态不成立后, 可基于空臂假设查询清空状态。旧的恢复条件为：

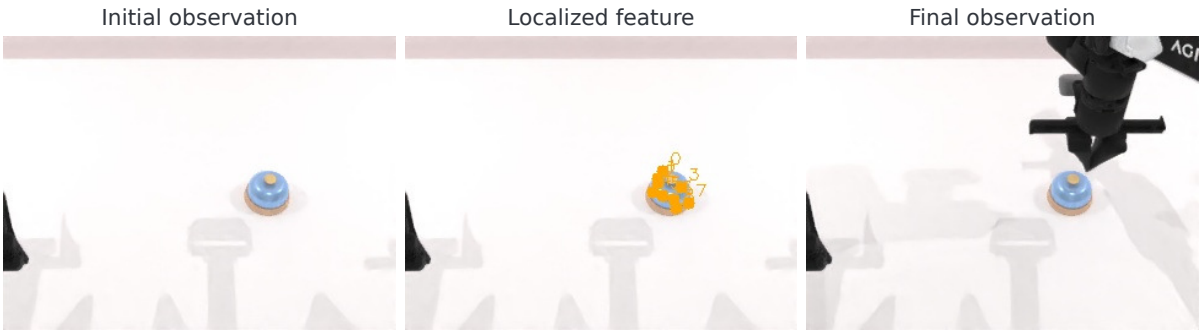


Figure 13 | 已发表的在成功 `click_bell` 开发试验中关于敲击技能的观察：初始视图、特征定位以及执行后的视图。结果由模拟器评分；图像大小为 320× 240 像素。

```
recovered = (
    recovery_move.get("ok") is True
    and recovery_measure.get("ok") is True
    and recovery_measure.get("pose_matches") is True
    and both_empty()
    and current_clear()
)
```

模型生成的修订版本在每次恢复尝试后评估终端占用情况，仅当确认两个机械臂均为空时才检查清除状态。它使用这些记录的检查结果来判断恢复是否成功。辅助条件和修订后的条件如下；中间的动作调用已省略。

```
def terminal_empty_clear():
    empty = both_empty()
    if not empty:
        return False, False
    return True, empty_current_clear()

recovery_empty, recovery_clear = terminal_empty_clear()
recovered = (
    recovery_move.get("ok") is True
    and recovery_measure.get("ok") is True
    and recovery_measure.get("pose_matches") is True
    and recovery_empty
    and recovery_clear
)
```

一位独立的经理评审接受了此版本，该版本在发布前通过了 2/3 的模拟器开发检查。之前的候选版本仅停留在代码审查阶段，未进行任何模拟器运行。完整补丁还更改了诊断信息，并移除了一个可选的视觉差异检查。一名操作员独立维护了摄像头、底层接口和监视器。代码片段展示了终端检查的修订内容。

D.4. 第 25 轮和第 33 轮的物理技能修订

两条记录均来自移动操作臂上的家庭清理任务（第 4.1 节），如图 3b 所示。每次修订均以触发运行、最近四个有效的运行作为经验池，以及更改的文件为依据；在验证完成前，实时迭代被暂停。

第二十五轮：放置观察与释放。 *执行.* 机器人将食物包装袋置于正确的目标垃圾箱前方，该垃圾箱与附近的浅蓝色桶在视觉上保持明显区分。编译后的观测移动报告了 0.052 m 和 17.1° 的容差偏差，尽管当前关节状态和末端执行器状态以及第三人称视频均显示机械臂已到达目标姿态。不受限的回退机制从同一视角重新执行了开合估计，调用了带有无效自检目标的机械臂探索，并耗尽了 28 次合并的工具调用。工作区探测已提前在运行时内存中存储了完整的三步释放序列，但模型传递给 `place_object` 的是不完整的副本。两次调用均在运动前失败关闭；夹爪从未打开，机器人最终返回至停靠位姿，物品仍被持握。

诊断. 最早的失败结点是放置观测移动的完成。其假负例扩展为重复的几何估计和不完整的释放计划。

修订. (i) 失败的观测移动仅接收一次只读姿态复核，而物理上已到达的姿态则无需重播放置链。(ii) 未达姿态向规划器返回一个不可释放的检查点，且工程师回退功能被禁用。(iii) 在容器上方进行探测和释放时，需从当前观测获取开合几何信息，而非仅依赖语义边缘标识。(iv) `place_object` 将模型写入的不完整序列替换为运行时目标内存中保存的完整序列。

验证. 五项新的回归测试、172 项放置测试、863 项管理器测试及 97 项真实机器人接口测试全部通过。下一次运行需要对场景进行全新的人工重置。

第三十三轮：目的地身份确认与导航交接。 *执行。*对五个航向进行一次扫描，成功记录了所有八个预期场景实体。第一个物品被释放并确认分离，但运行在约 25 分钟后结束，此时第二个物品仍处于持有状态。两次所有权验证失败。库存生产者选择了一个孤立但边缘裁剪的目的地桶视图，尽管同一扫描中包含了完整的视图；消费者随后将蓝色比例视为强制性检查项，拒绝了生产者认证的目的地。此前曾尝试导航至该物品，但未完成，且未生成导航后的图像；此状态被路由至工程师备用方案，该方案重复执行状态、电源和相机查询，在多次合并预算耗尽后，才通过一次编译重试成功。

*修订。*库存选择优先采用完整且非边缘裁剪的观测；在实例化过程中，查询颜色变为建议性检查，因为生产者拥有语义身份归属权，而当前帧的导航与操作检查仍然有效；无导航后图像的不完整导航将返回一个带类型的检查点给规划器，而非进入备用方案。此次修订未引入新的物体或目的地分支，未添加回合坐标，未设置历史姿态门限，也未改变全局颜色阈值。

*验证。*对触发回合的离线回放选择了完整的桶视图，实例化了目的地，并将不完整的导航路由至规划器。经验池中包含四次连续受阻的运行，分别因不同终端失败：由得分否决拒绝的真实目的地、防御性释放后检查、放置区域耗尽，以及上述的颜色否决。共通过 28 次聚焦测试、866 次管理器测试和 97 次真实机器人接口测试。回合前后的重置均已规划；执行过程中未发生任何

非计划的人工提示。

D.5. 需要人类知识的更正

在物理发展（第 4.1 节）中分类的 53 个修订触发器中有 4 个需要系统无法从自身记录中获取的知识（图 14）。在第 23 轮，一个已释放的物品落在了预期容器之外。操作员指出，确认的错位会污染场景，必须由人工重置，而不是作为机器人恢复的新源对象处理。在第 31 轮，抓取方式必须遵守默认的线性速度 0.02 m/s，而经过验证的动作序列尚未使用该值；操作员提供了并确认了这一安全限制。在第 37 轮，包裹在释放后掉落在箱子旁边，操作员判断此次放置为失败，需重置场景，而非视为成功或自动恢复的步骤。在第 99 轮，任务停止时机器人仍持有一个物品，随后在等待停靠期间该物品丢失。操作员指定，保持保护必须继承经审查的握持宽度和力度，当保持丢失时，应冻结运动并请求确认，而不是继续移动或释放。每次修正均作为可复用的合约、检查点或对相关技能的测试保留下来，因此无需再次提出相同提示。



(a) Round 23: item released outside the container

(b) Round 31: grasp approach under a speed limit

(c) Round 37: wrapper lands beside the bin

(d) Round 99: item still held after termination

Figure 14 | **需要人类知识的物理修正**。 (a) 确认错位后重置情景；(b) 操作员指定的安全抓取速度；(c) 包装物在料箱外释放；(d) 终止后的保持保护。